

DOCFLOW

Real-Time Collaborative Document Editor

FAANG-Level Engineering Project Specification

Complete Product, Engineering & System Design Document

Frontend: React 18 + TypeScript | Realtime: WebSockets + CRDT (Yjs)

Backend: Node.js · Go microservices · PostgreSQL · Redis · S3 · Kafka

Version 1.0 | Engineering Specification

1. Executive Summary

DocFlow is a production-grade, real-time collaborative document editor engineered to operate at the scale and quality bar of FAANG companies. It enables thousands of users to simultaneously author, edit, comment on, and review rich-text documents with sub-100ms latency for collaborative operations, full offline support, end-to-end version history, and granular permissioning.

This specification covers product features, user-facing screens, system architecture, data models, API contracts, real-time collaboration algorithms, infrastructure, observability, security, scaling strategy, project roadmap, and deliverables a senior engineer would be expected to produce. It is structured so that an engineering team can pick it up and execute, and an interview committee can evaluate the depth of design thinking.

1.1 Vision

Build the infrastructure that makes structured human collaboration on documents feel as fluid and inevitable as a single person typing alone.

1.2 Goals

- Real-time multi-user editing with sub-100ms perceived latency at p95.
- Strong consistency guarantees on document state across clients via CRDTs.
- Offline-first: full read/write while disconnected with seamless conflict-free reconciliation.
- Horizontal scalability to 10M+ documents and 100K concurrent editors.
- End-to-end encryption optional per workspace; SOC 2 + GDPR compliant by default.
- 99.99% availability SLO for the editing path.

1.3 Non-Goals

- Building a full Office suite (slides, sheets) — out of scope for v1.
- Native desktop apps — web-first; PWA covers offline use cases.
- On-premises deployment in v1; cloud-only initially.

1.4 Success Metrics

- p95 keystroke-to-broadcast latency under 80ms within a region.
- p99 document open time under 1.2s for documents up to 200 pages.
- DAU / MAU ratio > 0.55 within 6 months of GA.
- Edit session crash-free rate > 99.95%.
- Conflict resolution correctness: zero divergent states observed across reconciliation tests.

2. Product Features

Features are grouped by domain. Each feature lists user value, key behaviors, and notable engineering considerations. P0 features are required for GA; P1 ships in the first quarter post-GA; P2 is roadmap.

2.1 Editing & Formatting (P0)

- Rich text: bold, italic, underline, strikethrough, code, superscript, subscript, highlight, color.
- Block types: paragraph, headings H1–H6, bulleted list, numbered list, checklist, code block, blockquote, callout, divider.
- Inline elements: links, mentions (@user), inline comments anchor, emoji, math (LaTeX), inline images.
- Tables with merged cells, column resize, header rows, sorting.
- Images: drag-and-drop upload, resize handles, alt text, alignment, caption.
- Embeds: YouTube, Figma, Loom, Google Maps, Tweets — via oEmbed and a server-side proxy.
- Page-level: cover image, icon, title, subtitle, multi-column layouts.
- Markdown shortcuts: typing ``#``, `##``, ``-``, ``1.``, ``'''`` triggers block conversion.
- Slash command menu: type ``/`` to insert any block type with fuzzy search.
- Keyboard shortcuts: full Cmd/Ctrl coverage matching Google Docs / Notion conventions.

2.2 Real-Time Collaboration (P0)

- Multiple cursors with user-colored carets and live name labels.
- Live selection highlights showing what each collaborator has selected.
- Presence indicators: avatar stack with active/idle/away status.
- Follow mode: click a collaborator's avatar to lock viewport to their cursor.
- Typing indicator: subtle animation when a collaborator is actively editing a block.
- Offline editing: writes queued locally, replayed on reconnect; CRDT guarantees convergence.
- Soft-locking on shared embeds (e.g., a single-edit table) to reduce thrash.

2.3 Comments, Suggestions & Reviews (P0)

- Inline comments anchored to a text range; threaded replies; resolve / re-open.
- @-mentions with email + in-app notifications.
- Suggestion mode: edits appear as tracked changes that an authorized user can accept or reject.
- Comment filters: open, resolved, mentioning me, by author, by page.
- Reactions on comments (👍 👎 🤔 🙌).

2.4 Version History (P0)

- Automatic snapshots every 30 seconds during active editing, plus session-end snapshot.
- Manual named versions: "Pre-launch draft", "Legal review v2".
- Diff view: side-by-side and inline diff colored by author.
- Restore-as-current with one click; old version preserved.
- Per-version export to PDF, DOCX, Markdown, HTML.

2.5 Sharing & Permissions (P0)

- Per-document roles: Owner, Editor, Commenter, Viewer.
- Workspace roles: Admin, Member, Guest.
- Shareable links with expiration and password options.
- Domain-restricted sharing (e.g., anyone @company.com).
- Granular link access: "Anyone with link", "Only invited", "Workspace".
- Audit log of access and permission changes per document.

2.6 Search & Discovery (P0)

- Full-text search across all accessible documents with typo tolerance.
- Faceted filters: owner, last edited, has comments, contains image, document type.
- In-document find/replace with regex.
- Recently viewed and favorites.
- Workspace home: personalized feed of activity, mentions, due tasks.

2.7 Templates & Imports (P1)

- Template gallery: meeting notes, PRD, design doc, brainstorm, retro, OKRs.
- Import from .docx, .md, Google Docs, Notion export, HTML.
- Export to .docx, .pdf, .md, HTML, JSON.

2.8 AI Assistance (P1)

- Inline AI: rewrite, summarize, translate, fix grammar, change tone.
- Document Q&A: ask questions about the doc with cited spans.
- Auto-generated TL;DR at top of long documents.
- Smart formatting suggestions and auto-tag generation.

2.9 Notifications (P0)

- In-app notification center: mentions, replies, share invites, version restored.
- Email digests: hourly, daily, weekly cadences.
- Per-document subscription: notify on every edit, comment, or never.

2.10 Mobile & Offline (P1)

- Responsive React web app and PWA installable on iOS/Android.
- Offline read/write up to a configurable cache size (default 500 MB).
- Background sync via Service Worker when connection is restored.

2.11 Admin & Workspace Management (P0)

- Workspace settings: name, logo, default permissions, allowed domains.
- Member management: invite, role change, deactivate, transfer ownership.
- Billing: plans, seats, usage, invoices.

- Audit log: per-user, per-resource, exportable to SIEM.
- SSO via SAML 2.0 and OIDC; SCIM provisioning.

2.12 Compliance & Security (P0)

- AES-256 encryption at rest; TLS 1.3 in transit.
- Optional customer-managed keys (CMK) per workspace.
- GDPR data export and right-to-be-forgotten pipelines.
- SOC 2 Type II controls baked into infra defaults.
- Rate limiting, anomaly detection, suspicious-login alerts.

3. User Personas & Key Journeys

3.1 Personas

Persona	Role	Primary Needs
Priya, PM	Product Manager	Write PRDs with engineers and designers in real time, gather comments, track decisions.
Marco, Engineer	Senior Engineer	Design docs with diagrams and code snippets, version history, link to tickets.
Lin, Designer	Designer	Embed Figma frames, brainstorm with stickies, comment threads on visuals.
Sam, Exec	VP / Director	Read with side panel of comments, leave directional feedback, view diffs across versions.
Dana, Admin	IT / Workspace Admin	Provision users via SCIM, audit access, enforce DLP and retention policies.

3.2 Critical User Journeys

Journey A: Two collaborators editing simultaneously

1. Priya opens a PRD and shares the link with Marco.
2. Marco opens the link; Priya sees his avatar appear in the presence stack within 200ms.
3. Both type into different paragraphs. Each sees the other's caret, name label, and inserted characters within 80ms.
4. Marco loses Wi-Fi for 3 minutes, continues editing. Edits queue locally.
5. On reconnect, his queued operations are merged via CRDT; no conflicts surface to either user.
6. Priya leaves a comment @-mentioning Marco. He receives a notification.

Journey B: Reviewing a doc with suggestions

7. Sam opens a doc shared as Commenter.
8. Sam toggles Suggestion mode and edits a paragraph; changes appear as tracked insertions/deletions colored by author.
9. Priya, the Owner, reviews suggestions in a side panel and accepts or rejects each.
10. Accepted suggestions become canonical; doc-history records the transition.

Journey C: Recovering a deleted section

11. Marco opens version history.
12. Selects a version from yesterday.
13. Diff highlights deleted paragraph in red on the right.
14. Marco copies that paragraph and pastes back into the live doc.

4. Screens & UI Specification

The app is a single-page React 18 application with a small set of top-level routes. All screens follow a consistent design system (see §10). Each screen below lists purpose, key UI regions, primary interactions, and edge states.

4.1 Screen Inventory

#	Screen	Route	Purpose
1	Landing / Marketing	/	Public marketing page with sign-up CTA.
2	Sign Up / Login	/auth/*	Email + OAuth (Google, Microsoft, Apple) + SSO entry.
3	Onboarding	/welcome	Workspace creation, invite teammates, pick template.
4	Workspace Home	/home	Personalized feed: recent docs, mentions, activity.
5	Document List / Drive	/drive	Files & folders, breadcrumbs, sort, filter.
6	Document Editor	/d/:docId	The core real-time editor. Most engineering effort lives here.
7	Comments Panel	(drawer in editor)	Threaded comments, filters, resolve/reopen.
8	Version History	/d/:docId/history	Version list, side-by-side diff, restore.
9	Share Dialog	(modal)	Permissions, link settings, invite by email.
10	Search Results	/search?q=	Cross-doc search with facets and snippets.
11	Notifications	/notifications	Activity center: mentions, invites, system.
12	Profile & Account	/me	User profile, theme, language, sessions, security.
13	Workspace Settings	/settings/workspace	Members, billing, integrations, audit log.
14	Admin Console	/admin	SSO, SCIM, DLP, retention, security policies.
15	Templates Gallery	/templates	Browse and start from templates.
16	Trash	/trash	Recoverable deleted docs with TTL countdown.
17	Errors & Empty States	varies	404, 403, offline, no-docs, no-results, rate-limited.

4.2 Editor Screen — Detailed Specification

The editor is the heart of the product. It must remain responsive at 60fps on a 10K-paragraph document on mid-tier laptops.

Layout regions

- Top App Bar: doc title, breadcrumbs, share button, presence stack, more menu.
- Toolbar: formatting controls, block-type selector, insert menu (image, table, embed).
- Left Sidebar (collapsible): document outline (auto-generated from headings), pages tree.
- Main Canvas: the editable document; max-width container with comfortable line-length.
- Right Drawer (toggleable): comments, version history, AI assistant, properties.
- Status Bar (footer): connection status, last saved timestamp, word count, character count.

Key interactions

- Slash menu: ``/`` opens command palette; arrow keys + enter to insert.
- Floating selection toolbar appears on text selection with format and comment actions.
- Block handle (`:::`) on hover shows drag handle and block menu (duplicate, delete, transform).
- Drag-and-drop blocks anywhere in the document with smooth animation.
- Paste handling: rich (HTML), markdown, plain text, image — sanitized server-side.

Edge states

- Connection lost: amber banner, edits continue queued; CRDT log persists in IndexedDB.
- Permission downgraded mid-session: graceful read-only mode with toast.
- Conflict on a server-side guard (e.g., title length): inline error + revert option.
- Document over 500 pages: virtualized rendering kicks in.

4.3 Component Inventory

Approximate count of distinct, reusable React components.

Layer	Count	Examples
Atoms	~45	Button, Icon, Avatar, Input, Tooltip, Badge, Spinner, Toast
Molecules	~60	MenuItem, Toolbar, Tab, ListItem, ContextMenu, Popover
Organisms	~35	Editor, CommentThread, ShareDialog, VersionDiff, PresenceStack
Templates	~12	AppShell, EditorLayout, AdminLayout, AuthLayout
Pages	~17	WorkspaceHome, EditorPage, AdminConsole, etc.

5. Technology Stack

Choices below are justified against the requirements: real-time correctness, scalability, developer velocity, and operational maturity. Where a choice is non-obvious, the rationale is explicit.

5.1 Frontend

Layer	Choice	Rationale
Framework	React 18 + TypeScript	Concurrent rendering, Suspense, broad ecosystem, type safety.
Build	Vite + SWC	Sub-second HMR; faster than Webpack at scale.
Routing	React Router v6	File-based not required; explicit routes fit a SaaS app.
State	Zustand + Jotai	Zustand for global UI; Jotai for fine-grained atomic state in editor.
Data fetching	TanStack Query	Caching, retries, dedup; pairs with REST and tRPC.
Editor core	ProseMirror + Yjs	ProseMirror = best-in-class doc model; Yjs = mature CRDT.
Collab binding	y-prosemirror	Maps Yjs operations to ProseMirror transactions.
Styling	Tailwind CSS + CSS vars	Utility-first plus design tokens for theming.
UI primitives	Radix UI	Accessible, unstyled primitives composed with Tailwind.
Icons	Lucide React	Consistent line-icon set.
Forms	React Hook Form + Zod	Performant uncontrolled forms with schema validation.
Animations	Framer Motion	Layout animations and gesture support.
Charts	Recharts / D3	For analytics screens.
i18n	react-intl (ICU)	Plural and date formatting for 12 launch locales.
Testing	Vitest + RTL + Playwright	Unit, component, end-to-end.
Lint / Format	ESLint + Prettier + tsc	Strict typing; no implicit any.

5.2 Real-Time Collaboration

- Yjs as the CRDT runtime. Documents represented as Y.XmlFragment to map to ProseMirror nodes.
- y-websocket-style transport, but custom server in Go for awareness, auth, and persistence.
- Awareness channel for cursors, selections, and presence (separate from doc state).
- Snapshot-on-idle: server materializes the latest CRDT state to a Postgres BLOB every N updates.

- Garbage collection of tombstones on snapshot to keep document size bounded.

5.3 Backend Services

Service	Language	Responsibility
api-gateway	Node.js (Fastify)	Public REST + tRPC, auth, rate limiting, WAF integration.
collab-service	Go	WebSocket termination, awareness, CRDT update fanout.
document-service	Node.js (NestJS)	Document CRUD, versioning, snapshots, exports.
search-service	Go + OpenSearch	Full-text indexing, query, suggestions.
auth-service	Node.js	OAuth, SSO, sessions, MFA, SCIM.
notification-service	Node.js	In-app feed, email digests, push fanout.
billing-service	Node.js + Stripe	Plans, seats, invoices, dunning.
ai-service	Python (FastAPI)	LLM proxy, prompt orchestration, retrieval.
media-service	Go	Image uploads, thumbnailing, virus scan, CDN signing.
audit-service	Go	Append-only audit events, exports to SIEM.

5.4 Data Stores

Store	Use
PostgreSQL (RDS, Multi-AZ)	Source of truth for docs metadata, users, workspaces, permissions, billing.
Redis (ElastiCache)	Sessions, presence, awareness fanout, rate limits, hot caches.
S3	Document snapshots, attachments, exports, backups.
OpenSearch	Full-text doc search, audit log search.
ClickHouse	Product analytics, usage metrics, internal dashboards.
Kafka (MSK)	Event bus: doc.updated, comment.created, audit, billing.
FoundationDB (optional)	Global metadata for cross-region routing of CRDT shards.

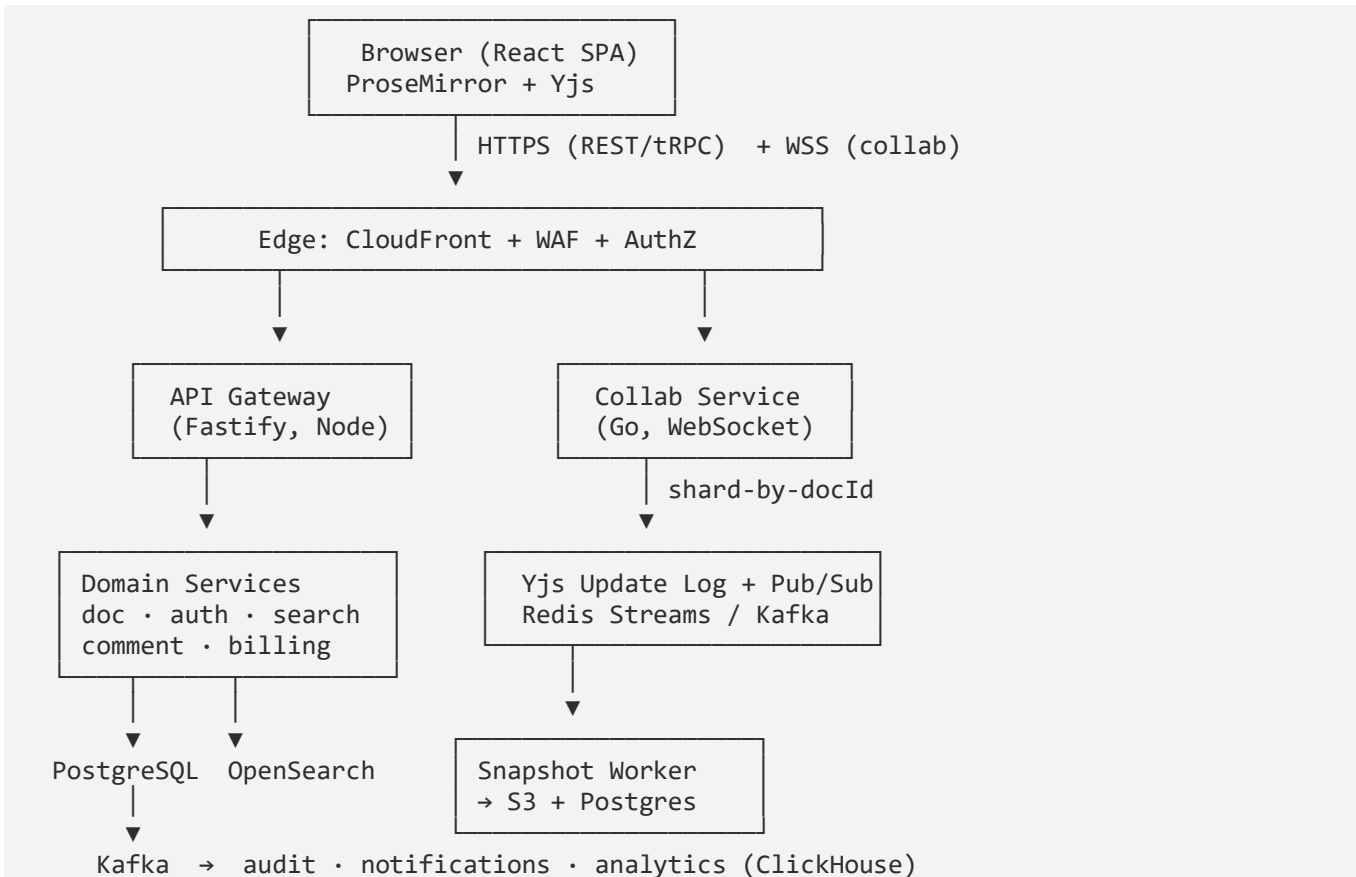
5.5 Infrastructure & DevOps

- Cloud: AWS primary (us-east-1, us-west-2, eu-west-1, ap-southeast-1).
- Compute: EKS (Kubernetes) for stateless services; EC2 ASG for collab nodes (sticky shards).
- Edge: CloudFront + Lambda@Edge for static assets and geo-routing.

- CI/CD: GitHub Actions → Argo CD; trunk-based development; feature flags via LaunchDarkly.
- IaC: Terraform modules per service; Atlantis for PR-driven plans.
- Observability: OpenTelemetry → Tempo (traces) + Prometheus (metrics) + Loki (logs); Grafana UIs.
- Error tracking: Sentry frontend + backend with source maps.
- Secrets: AWS Secrets Manager + IRSA on EKS; no long-lived keys in containers.
- Security: AWS WAF, GuardDuty, Inspector; per-PR SAST (Semgrep) and dependency scanning (Snyk).

6. System Architecture

6.1 High-Level Architecture (text diagram)



6.2 Real-Time Collaboration Flow

15. Client opens document. SPA fetches snapshot URL from document-service and downloads latest CRDT snapshot from S3 via signed URL.
16. Client connects WebSocket to collab-service at `wss://collab.docflow.app/d/:docId` with a short-lived JWT.
17. Collab-service authenticates, joins the shard for that docId, and sends any CRDT updates newer than the snapshot.
18. Client applies updates, then user begins editing. Each local Yjs update is sent to the server.
19. Server persists the update to a per-document append-only log (Redis Stream + Kafka) and broadcasts to all subscribers.
20. Awareness updates (cursor, selection, name) flow on a separate channel, are not persisted, and expire on disconnect.
21. Every N updates or T seconds, a snapshot worker rolls up the log into a new compacted snapshot in S3 and trims the log.
22. On disconnect, client buffers updates locally in IndexedDB. On reconnect, server reconciles by accepting the buffered updates.

6.3 Sharding & Scalability

- Documents are sharded by docId hash across collab-service instances. A consistent-hash ring with 256 virtual nodes routes WebSocket connections to the owning node.
- Sticky routing: client's WSS connection includes the shard hint; the load balancer forwards to the owning node, falling back to lookup via Redis if unavailable.
- Hot-doc protection: any document with > 5K concurrent editors is split-replicated with a primary shard and read-only mirrors; writes still serialize through the primary.
- Cross-region: CRDT shards are pinned to a home region; cross-region clients connect to the nearest edge and tunnel to the home shard. For multi-region authoring, async replication of compacted snapshots is sufficient since CRDTs converge.

6.4 Data Flow Examples

Save and version snapshot

23. User idles for 30s with unsaved updates.
24. Snapshot worker reads the doc's update log since last snapshot.
25. Builds a new compacted Yjs document state.
26. Writes to S3 (versioned bucket) and inserts a row in document_versions.
27. Trims the update log up to the snapshot offset.

Comment created

28. Client posts to POST /api/v1/comments.
29. API persists row in comments table; emits comment.created on Kafka.
30. Notification-service consumes the event, resolves @-mentions, fanouts in-app and email.
31. Collab-service consumes the event and pushes a metadata-only update to other clients in the doc.

7. Data Model

PostgreSQL is the source of truth for relational entities. Document content lives in S3 as CRDT snapshots, with the latest update log in Redis. Below are the core tables (simplified).

7.1 Core Tables

```

users (
  id UUID PK, email CITEXT UNIQUE, name TEXT, avatar_url TEXT,
  password_hash TEXT, mfa_secret TEXT, locale TEXT,
  created_at TIMESTAMPTZ, last_active_at TIMESTAMPTZ
)

workspaces (
  id UUID PK, name TEXT, slug TEXT UNIQUE, plan TEXT,
  owner_user_id UUID FK → users.id, created_at TIMESTAMPTZ
)

workspace_members (
  workspace_id UUID, user_id UUID, role TEXT,  -- admin|member|guest
  PRIMARY KEY (workspace_id, user_id)
)

documents (
  id UUID PK, workspace_id UUID FK, parent_folder_id UUID NULLABLE,
  title TEXT, icon TEXT, cover_url TEXT,
  created_by UUID, created_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ, last_editor_id UUID,
  deleted_at TIMESTAMPTZ NULL,  -- soft delete
  current_snapshot_key TEXT,    -- S3 key
  current_snapshot_version BIGINT
)

document_acl (
  document_id UUID, principal_type TEXT,  -- user|workspace|link
  principal_id UUID NULLABLE, role TEXT,  -- owner|editor|commenter|viewer
  expires_at TIMESTAMPTZ NULL
)

document_versions (
  id UUID PK, document_id UUID, version BIGINT,
  snapshot_key TEXT, created_by UUID,
  label TEXT NULLABLE, created_at TIMESTAMPTZ
)

comments (
  id UUID PK, document_id UUID, anchor JSONB,  -- prosemirror range
  thread_id UUID, parent_comment_id UUID NULLABLE,
  author_id UUID, body_md TEXT, body_json JSONB,
  resolved_at TIMESTAMPTZ NULL, created_at TIMESTAMPTZ
)

notifications (
  id UUID PK, recipient_id UUID, kind TEXT,
  payload JSONB, read_at TIMESTAMPTZ NULL,

```

```

    created_at TIMESTAMPTZ
)

audit_events (
  id BIGSERIAL PK, workspace_id UUID, actor_id UUID,
  action TEXT, resource_type TEXT, resource_id UUID,
  ip INET, ua TEXT, payload JSONB, occurred_at TIMESTAMPTZ
)

```

7.2 Indexes & Partitioning

- documents (workspace_id, updated_at DESC) — drive home feed.
- document_acl (document_id) and (principal_id) — permission checks both directions.
- comments (document_id, created_at) — load comments for a doc.
- audit_events partitioned monthly by occurred_at; older partitions archived to S3.
- notifications partitioned by recipient_id hash; read_at IS NULL fast index.

7.3 CRDT Storage

- Latest snapshot: S3 key `snapshots/{docId}/{version}.ybin` (Yjs binary update format).
- Update log: Redis Stream `doc:{docId}:log` capped by version offset.
- Durable log: same updates fanned to Kafka topic `crdt.updates` keyed by docId for replay and DR.
- Garbage collection: snapshot worker compacts updates → new snapshot every 500 ops or 60s idle.

8. API Design

8.1 REST + tRPC Surface

Public API uses REST with versioned URLs (/api/v1) for external integrations and OAuth apps. The first-party web client uses tRPC over HTTP for end-to-end type safety. Both share the same domain layer.

8.2 Selected Endpoints

Authentication

```
POST /api/v1/auth/signup
POST /api/v1/auth/login
POST /api/v1/auth/logout
POST /api/v1/auth/refresh
GET  /api/v1/auth/sso/:workspaceSlug
```

Workspaces

```
GET /api/v1/workspaces
POST /api/v1/workspaces
GET /api/v1/workspaces/:id/members
POST /api/v1/workspaces/:id/invite
```

Documents

```
GET /api/v1/documents?folderId=&cursor=
POST /api/v1/documents
GET /api/v1/documents/:id
PATCH /api/v1/documents/:id // metadata only
DELETE /api/v1/documents/:id // soft delete
GET /api/v1/documents/:id/snapshot // signed S3 URL
GET /api/v1/documents/:id/versions
POST /api/v1/documents/:id/versions/:vid/restore
```

Sharing & ACL

```
GET /api/v1/documents/:id/acl
PUT /api/v1/documents/:id/acl
POST /api/v1/documents/:id/links
```

Comments

```
GET /api/v1/documents/:id/comments
POST /api/v1/documents/:id/comments
PATCH /api/v1/comments/:cid
POST /api/v1/comments/:cid/resolve
```

Search

```
GET /api/v1/search?q=...&type=doc
```

Realtime (WebSocket)

```
WSS /collab/d/:docId // CRDT updates
WSS /collab/d/:docId/aware // awareness (cursors, selections)
```

8.3 Realtime Wire Protocol

Binary frames over WebSocket. Each frame begins with a 1-byte type tag.

Tag	Direction	Payload
0x01	C → S	Yjs CRDT update (variable length, Yjs binary format).
0x02	S → C	Yjs CRDT update broadcast from a peer.
0x03	C ↔ S	Awareness update (cursor, selection, presence).
0x04	S → C	Sync step 1 (state vector).
0x05	S → C	Sync step 2 (missing updates).
0x10	S → C	Server error / ack with code.
0xFF	C → S	Heartbeat ping (every 25s).

8.4 Auth & Authorization

- Access token: short-lived JWT (15 min) signed RS256, audience-bound per service.
- Refresh token: rotating, single-use, stored in HttpOnly Secure SameSite=Lax cookie.
- Per-document access checked at WebSocket upgrade via a signed scope token containing { docId, role, exp }.
- Role hierarchy: Owner > Editor > Commenter > Viewer; checked in a single permissions middleware.

8.5 Rate Limiting

- Per-IP and per-user token-bucket via Redis: 600 req/min default, burst 100.
- Realtime: max 200 ops/sec per client; excess is buffered then throttled.
- Heavier limits on auth and AI endpoints; 429 returned with Retry-After.

9. Real-Time Collaboration: CRDT Deep Dive

9.1 Why CRDT and not OT

Operational Transformation (OT) — used historically by Google Docs — requires a central authoritative server to transform operations against each other in a globally agreed order. CRDTs (Conflict-free Replicated Data Types) embed enough information in each operation that any two replicas converge to the same state without a central transform function. This makes offline editing, peer-to-peer fallbacks, and simpler server logic tractable. The trade-off is metadata overhead, which Yjs minimizes via integer client IDs and run-length encoding.

9.2 Document Representation

- Top-level Yjs document containing a Y.XmlFragment named 'prosemirror'.
- Each ProseMirror node maps to a Y.XmlElement; text nodes map to Y.XmlText with format attributes.
- Block attributes (heading level, list type) live on the XmlElement; inline marks (bold, italic) on the XmlText.
- y-prosemirror keeps the two models bidirectionally bound: a ProseMirror transaction yields a Yjs transaction and vice versa.

9.3 Update Lifecycle

32. User types a character. ProseMirror transaction emitted.
33. y-prosemirror translates the transaction into a Yjs operation on Y.XmlText.
34. Yjs produces a binary update encoding only the delta (Item structs with client clock).
35. Update is sent to server (frame 0x01) and applied locally to the IndexedDB persistence.
36. Server appends to log, broadcasts to peers (frame 0x02). Peers apply, merging into their local doc; ProseMirror updates.

9.4 Awareness vs State

- State updates are persisted and replayed on load.
- Awareness updates (cursor, selection, user metadata) live only in memory, propagated through a separate channel; they expire when a client disconnects.
- Mixing them at the protocol layer keeps state minimal and avoids accidental persistence of ephemeral data.

9.5 Snapshots & Garbage Collection

- Every 500 operations or 60 seconds of idle, a snapshot worker compacts the Yjs update log.
- Compaction merges adjacent operations and drops tombstoned items beyond all clients' state vectors (safe deletion).
- Snapshot is written to S3 and the update log is trimmed up to the compacted offset.
- All clients on reconnect either replay updates since their last known clock or download the latest snapshot if too far behind.

9.6 Conflict Resolution Examples

Concurrent insert at the same position

User A inserts "hello" at position 5; User B inserts "world" at position 5 simultaneously. Yjs orders them by client ID + clock to produce a deterministic interleaving — both clients converge to the same final string, e.g., "helloworld" or "worldhello" depending on IDs. Order is consistent across replicas.

Edit vs delete

User A bolds a word while User B deletes the same word. Yjs preserves the deletion as a tombstone; the bold mark applies to a tombstoned text and effectively no-ops. Both replicas show the word deleted.

Offline merge

User A is offline for 30 minutes editing chapter 1. User B online edits chapter 2. On reconnect, A's queued operations are sent and merged. Because Yjs operations are commutative, no transformation is needed; the merged state is identical to what both would have seen had they been online together.

9.7 Pitfalls and Mitigations

- Metadata bloat: bounded by snapshot+compaction; we measure update log size per doc and alert at 25 MB pre-snapshot.
- Long-lived offline edits: enforce a 14-day client-side staleness window; older offline edits require user resolution.
- Schema migrations: ProseMirror schema changes are forward-compatible; unknown nodes preserved as opaque; migrations applied by snapshot worker on read.

10. Design System

10.1 Tokens

- Color: 11-stop ramps for gray, brand, success, warn, danger; semantic tokens map them to surface, text, border.
- Typography: Inter for UI, JetBrains Mono for code, system serif for document body. 8 size steps; 1.5 line-height for body, 1.2 for headings.
- Spacing: 4px base; scale 4, 8, 12, 16, 24, 32, 48, 64.
- Radius: 4, 6, 8, 12, 16.
- Elevation: 5 shadow levels using layered RGBA shadows.
- Motion: 120ms (instant), 200ms (default), 320ms (large); ease-out for enter, ease-in for exit.

10.2 Theming

- Light, dark, and high-contrast themes via CSS variables on :root.
- Per-user preference synced across devices through user-settings service.
- Reduced-motion respected via prefers-reduced-motion.

10.3 Accessibility

- WCAG 2.1 AA target. Color contrast $\geq 4.5:1$ for body text, $3:1$ for UI.
- Full keyboard navigation; focus rings always visible (no outline removal).
- Screen reader support: ARIA roles on editor, live regions for collaborator presence and saving status.
- Localized to 12 languages at GA; RTL support for Arabic and Hebrew.

10.4 Component Documentation

- Storybook with controls, accessibility audit (axe), interaction tests.
- Each component documents props, do/don't usage, and dark-mode preview.

11. Performance Targets & Strategies

11.1 Targets

Metric	Target	Measurement
Time to interactive (cold)	$\leq 1.8s$ p75	RUM via web-vitals on home and editor.
Doc open (200 pages)	$\leq 1.2s$ p99	From route mount to first interaction-ready frame.
Keystroke → broadcast	$\leq 80ms$ p95	Client emit timestamp vs ack from server.
Broadcast → peer render	$\leq 120ms$ p95	Server emit timestamp vs peer apply.
Editor frame budget	≥ 60 fps	Long-task observer + custom RAF instrumentation.
Memory usage (10K-block doc)	≤ 280 MB	performance.memory + heap snapshots.

11.2 Frontend Strategies

- Virtualized rendering of blocks below the viewport for large documents.
- Code splitting per route and per heavy feature (AI panel, version diff, image cropper).
- Fine-grained reactivity with Jotai atoms scoped to blocks; re-render only edited block.
- Web Workers for syntax highlighting, markdown parsing, and CRDT decoding off the main thread.
- requestIdleCallback for non-critical work (autosave indicator, mention scanning).
- Image pipeline: server-side responsive variants and AVIF/WebP; lazy-load with width/height to avoid CLS.
- Service Worker pre-caches the editor shell and last 20 documents.

11.3 Backend Strategies

- Sticky WebSocket routing eliminates cross-node fanout for the common case.
- Redis pipelining for awareness and presence updates.
- Postgres read replicas for list/search APIs; primary for writes.
- CDN at the edge for snapshots; clients pull deltas from collab-service.
- Batching: Yjs server batches multiple updates within a 5ms window before broadcasting.

12. Security

12.1 Authentication

- Argon2id password hashing with memory cost 64 MB, time cost 3, parallelism 1.
- MFA: TOTP and WebAuthn; required for admins.
- Sessions tied to device fingerprints; suspicious-login email + step-up MFA.
- OAuth providers: Google, Microsoft, Apple, GitHub. SAML 2.0 + OIDC for enterprise SSO.
- SCIM 2.0 for user provisioning and deprovisioning.

12.2 Authorization

- RBAC at workspace level; ABAC for documents and folders combining role + link policies.
- All authorization checks centralized in a `permissions` package; no ad-hoc checks in handlers.
- Tested with a property-based test suite that asserts invariants (e.g., "viewer cannot mutate state").

12.3 Data Protection

- AES-256-GCM at rest; TLS 1.3 in transit; HSTS with preload.
- Per-workspace KMS key (CMK option for Enterprise).
- Field-level encryption for PII (email, IP) using an envelope key from KMS.
- Backups encrypted; restore drills quarterly.

12.4 Application Security

- Strict CSP with nonces; no inline event handlers; Trusted Types for DOM sinks.
- All HTML paste sanitized with DOMPurify (allowlist) on the client and a server-side re-validation.
- CSRF: token-bound mutations + SameSite=Lax cookies; double-submit pattern for cross-site flows.
- Dependency scanning (Snyk), SAST (Semgrep), DAST (ZAP) in CI; high-severity findings block merge.
- Bug bounty program with HackerOne post-GA.

12.5 Compliance

- SOC 2 Type II at GA + 12 months.
- GDPR: DPA, lawful basis, data residency in EU for EU customers, DSR pipelines.
- HIPAA-ready architecture path for healthcare customers (BAA + segregated tenancy).
- Quarterly penetration tests by an external firm.

13. Observability & Reliability

13.1 Telemetry

- OpenTelemetry SDK in all services; trace context propagated across HTTP and WSS.
- Logs: structured JSON, indexed in Loki; retention 30 days hot, 1 year cold in S3.
- Metrics: Prometheus with USE and RED dashboards per service.
- Frontend RUM: web-vitals + custom event sampling at 5%.

13.2 SLOs

Service / Path	SLO	Error Budget Policy
Editor read path	99.99% / 30d	Burn 25% in 24h triggers freeze on risky changes.
Realtime collab	99.95% / 30d	Same.
API mutations	99.9% / 30d	Page on-call below 99.5% in 1h.
Search	99.9% / 30d	Async; degrade to recent-only fallback.

13.3 Incident Response

- PagerDuty rotations per service; runbooks in repo with copy-paste commands.
- Severities: SEV1 (customer-impacting outage), SEV2 (degradation), SEV3 (single-customer).
- Blameless postmortems within 72h of any SEV1/2; action items tracked to closure.
- Game days quarterly: chaos experiments on collab-service partition, Redis failover, S3 throttling.

13.4 Disaster Recovery

- RPO $\leq$ 5 minutes, RTO $\leq$ 30 minutes for editing path.
- Cross-region S3 replication for snapshots.
- Postgres point-in-time recovery enabled; logical replicas in DR region.
- Quarterly DR drills with full failover and traffic cutover.

14. Testing Strategy

14.1 Test Pyramid

- Unit (~70%): pure functions, reducers, permission rules, CRDT helpers.
- Component (~20%): React components via Vitest + Testing Library; visual regression with Chromatic.
- E2E (~10%): Playwright covering critical journeys across Chromium, WebKit, Firefox.

14.2 Specialized Tests

- Concurrency tests: simulate N clients editing in parallel; assert convergence.
- Property-based tests for permissions and CRDT (fast-check).
- Load tests with k6 against staging: 100K concurrent WebSockets, 1M ops/min sustained.
- Chaos tests: kill collab nodes mid-session, partition Redis, throttle S3; assert no data loss.
- Accessibility tests: axe in unit tests; manual screen reader audits per release.

14.3 Pre-Production Gates

- All PRs: lint, typecheck, unit, component, contract tests.
- Main branch: full E2E suite; canary deploy to 1% of traffic.
- Release: error budget check; on-call signoff; staged rollout to 10% → 50% → 100%.

15. Project Roadmap

15.1 Phases

Phase	Duration	Scope
P0: Foundations	Weeks 1–4	Repo, CI/CD, design tokens, auth, workspaces, basic editor (no collab).
P1: Editor	Weeks 5–8	Rich text, blocks, slash menu, keyboard shortcuts, autosave.
P2: Collaboration	Weeks 9–14	Yjs integration, WebSocket server, awareness, presence, offline.
P3: Comments + Share	Weeks 15–18	Comments, suggestions, share dialog, link permissions.
P4: Versioning	Weeks 19–21	Snapshots, history UI, diff, restore.
P5: Search + Mobile	Weeks 22–24	OpenSearch indexing, search UI, PWA polish.
P6: Admin + SSO	Weeks 25–27	Workspace admin, SAML/OIDC, SCIM, audit log.
P7: Hardening + GA	Weeks 28–32	Perf, a11y, security audits, SOC 2 readiness, GA launch.

15.2 Team Shape (recommended)

- 1 Engineering Manager.
- 2 Frontend engineers (editor specialists).
- 2 Frontend engineers (product surfaces).
- 3 Backend engineers (services + realtime).
- 1 Infra/SRE engineer.
- 1 Designer.
- 1 Product Manager.
- 0.5 Security engineer (shared).

15.3 Risk Register

Risk	Severity	Mitigation
CRDT metadata bloat on hot docs	High	Aggressive snapshot+GC; per-doc size alerts.
Editor fps degradation at scale	High	Block virtualization; perf budgets enforced in CI.

Risk	Severity	Mitigation
Cross-tab collab anomalies	Medium	BroadcastChannel sync; conformance test suite.
WebSocket infra cost	Medium	Compression; awareness throttling; sticky LB.
Vendor lock-in on Yjs	Low	CRDT logic isolated behind a `Doc` interface.

16. Deliverables (What 'Done' Looks Like)

For a portfolio or interview-grade project, ship these artifacts. Each one signals a specific competency.

16.1 Code

- Monorepo (pnpm + Turborepo) with `apps/web`, `apps/collab`, `apps/api`, `packages/editor`, `packages/ui`, `packages/permissions`.
- CI: lint + typecheck + tests + build under 10 minutes.
- Docker images per service; docker-compose for local; Kind cluster manifests for k8s parity.

16.2 Documentation

- README with architecture diagram and a runnable quickstart.
- ADRs (Architecture Decision Records) for: CRDT vs OT, Yjs vs Automerge, REST vs tRPC vs GraphQL, single vs multi-tenant DB.
- API reference (OpenAPI 3.1) and Storybook published to GitHub Pages.
- This spec, kept in `/docs/spec.md`, updated with each major change.

16.3 Demos

- Public demo deployed to a custom domain with sample workspaces and seeded data.
- Loom walkthroughs (≤ 4 minutes each): editor demo, collaboration demo, admin tour.
- Performance benchmark report with real numbers from a load test.

16.4 Optional Stretch

- End-to-end encryption mode using per-doc symmetric keys with key wrapping in workspace KMS.
- Real-time co-editing on mobile via React Native sharing the same `packages/editor` core.
- AI agents that join as collaborators (cursor + edits) via the same WSS protocol.

17. Glossary

Term	Definition
CRDT	Conflict-free Replicated Data Type. A data structure where any replica can be updated independently and concurrently, and merged without a central coordinator.
Yjs	A high-performance CRDT framework written in JS, the engine behind y-prosemirror.
ProseMirror	A toolkit for building rich text editors with a strict, schema-defined document model.
Awareness	Ephemeral state about who is viewing a document and what they are doing (cursor, selection, presence).
Snapshot	A compacted representation of a document's state at a point in time, written to S3.
Sticky shard	A WebSocket routing strategy where all connections for a given document land on the same server.
Suggestion mode	An editing mode where changes appear as tracked insertions and deletions until accepted by an authorized user.
Workspace	A tenant boundary; users, documents, and billing are scoped to a workspace.
RPO / RTO	Recovery Point Objective (data loss tolerance) and Recovery Time Objective (downtime tolerance).

End of Specification